

Custom Object Detection with Yolov26

Step-by-Step Development using Ultralytics, Roboflow, and Google Colab

Francisco Javier Aguayo Hernández

ÍNDICE

ÍNDICE 1
1. INTRODUCTION 2
2. THEORETICAL FRAMEWORK 2
2.1 What is YOLOv26? 2
2.2 Models and Metrics 2
3. TRANSFER LEARNING 3
PHASE 1: DATASET PREPARATION 3
Image Capture and Labeling 3
Dataset Splitting 3
PHASE 2: GOOGLE COLAB SETUP 3
PHASE 3: MODEL TRAINING 4
PHASE 4: RESULTS ANALYSIS 4
PHASE 5: INFERENCE 4
4. CONCLUSIONS AND RECOMMENDATIONS 4

1. INTRODUCTION

The main objective of this project is to overcome the limitations of standard pre-trained models, such as the COCO Dataset (which includes only 80 generic categories), to detect specific objects defined by the user.

In this practical case, we selected three empty plastic boxes with different forms and colours. This document details the complete professional workflow, from initial data acquisition and manual labeling to real-time inference using high-performance deep learning algorithms.

2. THEORETICAL FRAMEWORK

2.1 What is YOLOv26?

YOLOv26 is the latest state-of-the-art iteration of the "You Only Look Once" family of models developed by Ultralytics. Its primary innovation is the ability to perform object localization and classification in a single forward pass through the neural network, making it significantly faster than previous architectures.

2.2 Models and Metrics

The project utilized specific versions of the YOLOv26 architecture as seen in the development environment:

- **yolo26n.pt (Nano):** Optimized for mobile and IoT devices.
- **yolo26m.pt (Medium):** A more accurate model requiring more powerful hardware.

Key Metrics:

- **mAP (Mean Average Precision):** Measures the accuracy of the bounding box overlap and class prediction.
- **FLOPS:** Measures the computational complexity of the model.

3. TRANSFER LEARNING

Training a deep neural network from scratch requires millions of images and massive computing power. To solve this, we use **Transfer Learning**. We take a model that already "knows" how to see (pre-trained on the COCO dataset) and "fine-tune" its last layers so it learns to recognize our specific toys. This allows us to achieve professional results with as few as 50-100 images per class.

PHASE 1: DATASET PREPARATION

Image Capture and Labeling

The success of an AI model depends 80% on the data. We captured photos of the three characters using different:

- **Angles:** Frontal, top-down, and perspective views.
- **Lighting:** Natural light, shadows, and artificial light.
- **Backgrounds:** Cluttered environments and neutral surfaces.

We used **Roboflow** to draw "Bounding Boxes" around each object, assigning the labels: `GreenCircle`, `BrownCircle`, and `GreenSquare`.



Collage

Dataset Splitting

To ensure the model generalizes well, the data was split into:

- **Training (70%):** Data the model learns from.
- **Validation (20%):** Data used to tune the model during training.
- **Testing (10%):** Final "exam" for the model with images it has never seen.

PHASE 2: GOOGLE COLAB SETUP

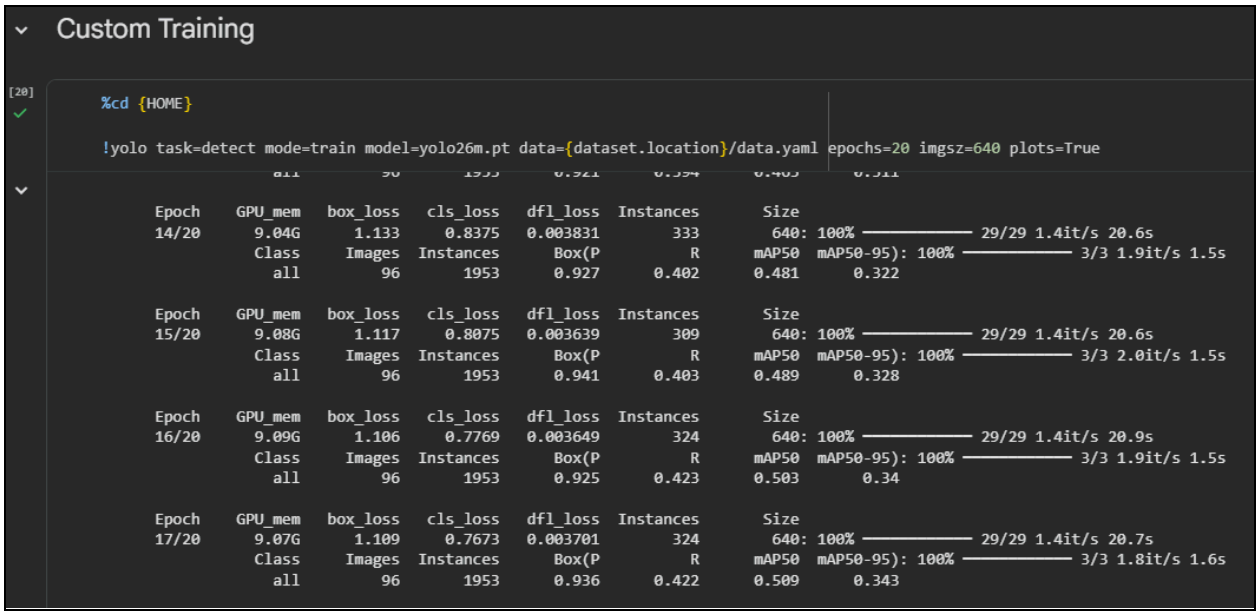
We utilized **Google Colab** to access NVIDIA T4 GPUs for free.

1. **Runtime:** Set to T4 GPU.
2. **Libraries:** Installed via pip install ultralytics roboflow.
3. **Environment:** The workspace included project files such as `yolo26m.pt`, `yolo26n.pt`, and test images like `dog-2.jpeg`.

PHASE 3: MODEL TRAINING

The training process requires a data.yaml file to map images and labels.

- **Model Selection:** While the initial plan considered YOLOv11s, the environment was configured for **YOLOv26** models.
- **Epochs:** 15.
- **Image Size:** 640 pixels.

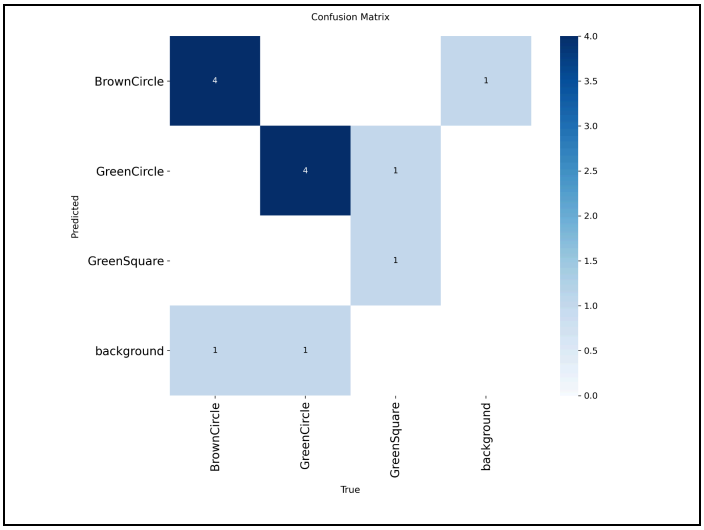


Custom Training

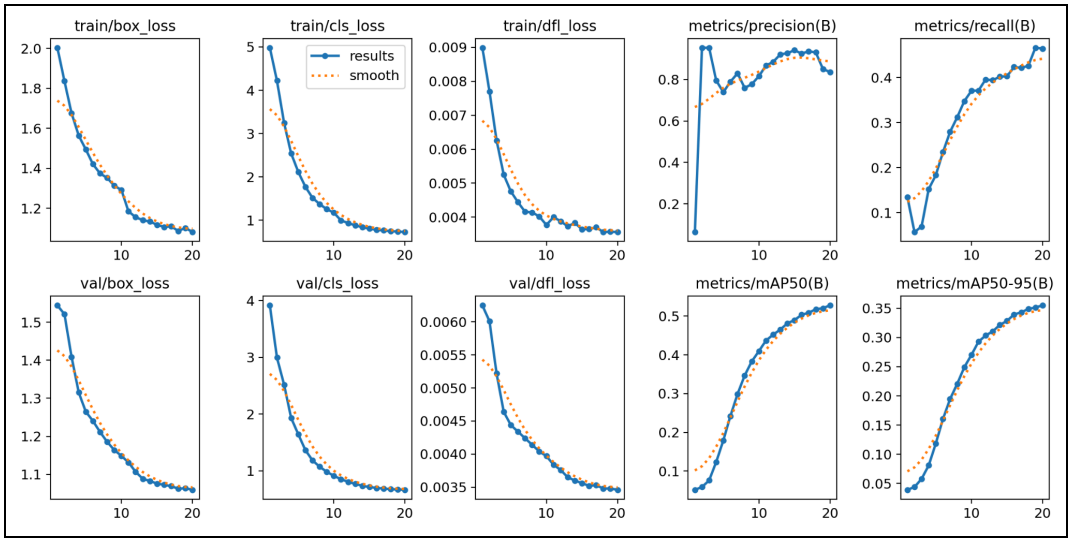
PHASE 4: RESULTS ANALYSIS

After training, the model generates evaluation files accessible via the IPython.display library:

- **Results Visualization:** results.png is generated in the runs/detect/train/ directory to track performance.
- **Confusion Matrix:** Shows if the model is confusing specific classes.
- **F1-Curve & Loss Curves:** Used to monitor the balance between precision and recall and to prevent **Overfitting**.



Confusion Matrix



F1-Curve

PHASE 5: INFERENCE

Once the training was complete, we used the `best.pt` file to perform:

1. **Static Testing:** Running the model on the 10% test images.
2. **Live Video:** Integrating with **OpenCV** to use the computer's webcam. The model successfully tracked the three toys in real-time with high confidence scores (usually above 0.85).

4. CONCLUSIONS AND RECOMMENDATIONS

- **Data Diversity:** The model performs best when trained with images similar to the final environment.
- **Scalability:** YOLOv26 can be exported to **ONNX** or **TensorRT** for edge devices.
- **Future Work:** Increasing the epoch count and using more "Augmentation" techniques could further improve accuracy.